Steven Francis
SDEV120 Computing Logic
Ivy Tech Community College
October 31, 2026

Jeopardy Game Plan

Module 5 Exercise 1 Plan

OVERVIEW

The program is a text based Jeopardy game that runs on one computer. Before drawing anything, I wanted to pin down the rules the program has to follow, the data it needs to keep track of, and the order everything happens in. Those three things drive the flowchart, so getting them right here saves rework later.

ROLES

Moderator (the program)

I planned the game from the moderator side, since the program has to make every call a host would make on the show. It shows the board, reads each clue, runs the timers, judges the responses, and keeps score.

Contestants

One to three people play on the same computer, and each one gets a buzzer key on the keyboard (A, G, and L). Contestants pick clues when they have control of the board, buzz in, and type their responses.

GAME COMPONENTS

Board

6 categories with 5 clues each (30 clues total). Clue values are $200, $400, $600, $800, and $1,000.

Clue file

Holds every category name, clue, and correct response, plus the Final Jeopardy category, clue, and response. Changing the categories only means editing the file, not the program.

Response format

A response only counts if it is phrased as a question, starting with "What is", "Who is", "Where is", "What are", or "Who are". The program strips those words and compares the rest to the stored answer, ignoring upper/lowercase. Leaving out the question words counts as wrong, and the player gets a reminder.

Timing

Players get 5 seconds after a clue is read to buzz in (BUZZ_TIME), 10 seconds to respond after buzzing (ANSWER_TIME), and 30 seconds for Final Jeopardy (FINAL_TIME). Running out of time counts as a blank, wrong response.

Scoring

A correct response adds the clue value and gives that player control of the board. A wrong response subtracts the value. If nobody buzzes in, no scores change. In Final Jeopardy, only players above $0 take part, and each one wagers between $0 and their current score.

DATA

Game data that several modules share is declared once in main(). Anything only one module needs (like a loop counter) is declared inside that module instead.

Name	Type	What it holds
MAX_PLAYERS, CATEGORIES, CLUES_PER_CAT	num constants	3 players, 6 categories, 5 clues per category
BUZZ_TIME, ANSWER_TIME, FINAL_TIME	num constants	5, 10, and 30 seconds
values[ ]	num array	200, 400, 600, 800, 1000
categories[ ], clues[ ][ ], responses[ ][ ]	string arrays	The board, loaded from the clue file
used[ ][ ]	string array	"Y" once a clue has been played, otherwise "N"
names[ ], scores[ ], wagers[ ]	arrays	One entry for each player
tried[ ]	string array	Whether each player already responded to the current clue
numPlayers, control, cluesLeft	num	Players in the game, who picks next, and clues remaining
cat, row, value, buzzer, tries	num	The clue being played and who buzzed in
pickOK, clueOpen, answered, tie	string flags	"Y" or "N" values that control loops and decisions

GAME FLOW

Step 1 **Get the players.** Ask how many people are playing and keep asking until the answer is 1, 2, or 3. Get each name and set every score to 0. Player 1 starts with control of the board.

Step 2 **Load the board.** Read the 6 categories, 30 clues, and 30 correct responses from the clue file, plus the Final Jeopardy clue. Mark every clue as unused and set cluesLeft to 30.

Step 3 **Play the round.** While cluesLeft is greater than 0, repeat the following.

- Show the board with the categories and the values that are still open.
- The player in control picks a category (1 to 6) and a clue (1 to 5). If the pick is off the board or already played, explain why and ask again.
- Read the category, value, and clue, then start the 5 second buzz timer.
- The first player to buzz in gets to respond, and each player only gets one try per clue. A correct response adds the value, gives that player control, and closes the clue. A wrong response subtracts the value and opens it back up for the other players.
- The clue also closes if nobody buzzes in time or every player has already tried. If nobody got it, show the correct response.
- Mark the clue as used and subtract 1 from cluesLeft.

Step 4 **Final Jeopardy.** Show the category. Every player above $0 enters a wager, and the program keeps asking until it falls between $0 and that player's score (players at $0 or below sit out). Show the clue, give each player 30 seconds to respond, then add or subtract each wager.

Step 5 **Show the results.** List each player's final score and announce the champion, or a tie if the top score is shared.

Step 6 **Stop.** The game ends.

MODULES

Any job that would otherwise show up in more than one place gets its own module. The clearest example is **checkResponse()**. Regular clues and Final Jeopardy both need to check that a response is phrased as a question and matches the answer, so I wrote that logic once and call it from both places instead of drawing it twice.

Module	Job	Called by
main()	Declares the game data, runs the round loop, then Final Jeopardy and the results	Starts the program
getPlayers()	Gets and checks the number of players, gets names, sets scores to 0	main()
loadClues()	Reads the board and the Final Jeopardy clue from the clue file	main()
selectClue()	Shows the board and gets a valid pick that has not been played	main()

Module	Job	Called by
playClue()	Reads the clue and runs the buzz in loop until the clue closes	main()
takeTurn()	Gets one player's response, judges it, and updates the score	playClue()
checkResponse()	Returns "Y" when a response is phrased as a question and is correct	takeTurn(), finalJeopardy()
getWagers()	Gets a valid wager from every player above $0	finalJeopardy()
finalJeopardy()	Plays the Final Jeopardy clue and applies the wagers	main()
showResults()	Shows the final scores and the champion or a tie	main()

I treated `waitForBuzz(BUZZ_TIME, tried)` as a built in timer function, the same way random() was treated in Module 4. It returns the number of the first player who presses their key in time and has not tried yet, or -1 if nobody does.

STRUCTURES AND FLAGS

Everything in the plan fits into the three basic structures. **Sequence** covers the straight runs of steps inside each module (like reading a clue and then starting the timer). **Selection** handles judging responses, checking picks and wagers, and deciding the winner. **Loops** run the main round, the buzz in process, every input check, and every pass over the players or the board.

I also used flag variables (pickOK, clueOpen, answered, and tie) so each loop has one clear test at the top and one exit, instead of jumping out from the middle.

UPDATES MADE WHILE FLOWCHARTING

The plan changed a bit once I started building the flowchart, which is pretty much what the assignment says to expect.

- playClue() was getting long, so I moved the response, judging, and scoring steps into their own module, takeTurn().
- Setting up the game was split into getPlayers() and loadClues() so each chart does one job and fits on one page.
- Getting wagers moved out of finalJeopardy() into getWagers(), since it has its own input check loop.
- I added the clueOpen flag so the buzz in loop ends cleanly when nobody buzzes in or someone answers correctly.

Daily Doubles and a Double Jeopardy round are left out of this version. Both could be added later as new modules without changing the structure above.